

—— // AWKRS // ——

awkrs – Language Reference

v0.4.21

LANGUAGE REFERENCE

every builtin · function · variable · special file



MENKE
TECHNOLOGIES

Copyright and Attribution

awkrs — awk as a fusevm bytecode engine

Copyright © 2026 MenkeTechnologies. All rights reserved.

No part of this book may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of the publisher, except for brief quotations in reviews.

Published by MenkeTechnologies.

The trademarks and product names referenced in this book are the property of their respective owners. This book is an independent work: it is not affiliated with, authorized by, sponsored by, or otherwise endorsed by the GNU Project, the gawk maintainers, the Rust Foundation, or any other project named in these pages.

The software this book describes

`awkrs` is released under the MIT License, Copyright © 2026 MenkeTechnologies. The source is at github.com/MenkeTechnologies/awkrs.

The language this book's software implements

`awkrs` is an independent implementation of the `awk` language: it parses `awk` source and lowers it to `fusevm` bytecode. It is not a fork of any existing `awk` and it incorporates no upstream `awk` source. The parity suites under `parity/` compare `awkrs`'s output against the behaviour of POSIX `awk` and of `gawk`; those programs are the work of their own authors and carry their own licenses. Where this book quotes their behaviour it is naming what `awkrs` was validated against.

The engine underneath

`awkrs` executes on `fusevm` — the bytecode virtual machine and three-tier Cranelift JIT shared across the language fleet. `fusevm` is released under the MIT License, Copyright © 2026 MenkeTechnologies, and its source is at github.com/MenkeTechnologies/fusevm.



MT PUBLISHING



MT OPEN SOURCE

Contents

Copyright and Attribution	1
String Functions	3
Arithmetic Functions	5
I/O and General Functions	8
Time Functions	10
Bitwise Functions	12
Array and Type Functions	14
Character and Text Functions	16
File and Directory Functions	17
Localization Functions	20
The Intercept Engine	21
Intercept Functions	23
Special Variables	24
Keywords and Control Flow	27
Pattern Forms	30
Operators	31
Redirection and getline Forms	35
printf Conversions	37
printf Flags, Width, and Precision	39
String Escape Sequences	41
Source Directives and Inline Rust	43
PROCINFO Keys	44
Array Traversal Order	48
Command-Line Options	50
Environment Variables	56

String Functions

11 topics

length

```
length([s])
```

Length of string *s* (or *\$0*), or element count of an array.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkLength` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

substr

```
substr(s, m [, n])
```

n-char substring of *s* starting at position *m* (1-based).

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkSubstr` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

index

```
index(s, t)
```

1-based position of *t* in *s*, or 0 if not found.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkIndex` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

split

```
split(s, arr [, fs [, seps]])
```

Split *s* into *arr* on *fs*; returns the field count.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

sub

```
sub(re, repl [, target])
```

Replace first match of *re* in *target* (*\$0*); returns 1/0.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkSub` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

gsub

```
gsub(re, repl [, target])
```

Replace all matches of `re` in `target` (`$0`); returns count.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkGsub` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

gensub

```
gensub(re, repl, how [, target])
```

Non-destructive global/Nth substitution returning the new string.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

match

```
match(s, re [, arr])
```

Set RSTART/RLENGTH to the match of `re` in `s`; returns position or 0.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkMatch` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

sprintf

```
sprintf(fmt, ...)
```

Format the arguments per `fmt` and return the string.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkSprintf` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

tolower

```
tolower(s)
```

Copy of `s` with uppercase letters lowercased.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkToLower` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

toupper

```
toupper(s)
```

Copy of `s` with lowercase letters uppercased.

Engine. Dispatched by awkrs's own VM. Lowered to the native `fusevm::Op::AwkToUpper` by the `AWKRS_FUSEVM_NATIVE=1` backend. The default numeric-chunk offload does not admit it, so an ordinary run always executes the VM path.

Arithmetic Functions

12 topics

sin

```
sin(x)
```

Sine of x (radians).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkSin` on both fusevm routes — the default numeric-chunk offload and the `AWKRS_FUSEVM_NATIVE=1` backend.

cos

```
cos(x)
```

Cosine of x (radians).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkCos` on both fusevm routes — the default numeric-chunk offload and the `AWKRS_FUSEVM_NATIVE=1` backend.

atan2

```
atan2(y, x)
```

Arctangent of y/x in radians.

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkAtan2` on both fusevm routes — the default numeric-chunk offload and the `AWKRS_FUSEVM_NATIVE=1` backend.

exp

```
exp(x)
```

e raised to the power x .

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkExp` on both fusevm routes — the default numeric-chunk offload and the `AWKRS_FUSEVM_NATIVE=1` backend.

log

```
log(x)
```

Natural logarithm of x .

Engine. Dispatched by awkrs's own VM. Also lowered to native fusevm ops on both routes: `fusevm::Op::AwkLogJit` by the default numeric-chunk offload, `fusevm::Op::AwkLog` by the `AWKRS_FUSEVM_NATIVE=1` backend.

sqrt

```
sqrt(x)
```

Square root of x .

Engine. Dispatched by awkrs's own VM. Also lowered to native fusevm ops on both routes: `fusevm::Op::AwkSqrtJit` by the default numeric-chunk offload, `fusevm::Op::AwkSqrt` by the `AWKRS_FUSEVM_NATIVE=1` backend.

int

```
int(x)
```

Integer part of `x`, truncated toward zero.

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkInt` on both fusevm routes — the default numeric-chunk offload and the `AWKRS_FUSEVM_NATIVE=1` backend.

intdiv

```
intdiv(a, b)
```

Truncating integer quotient of `a / b`; a zero divisor is a fatal error. Diverges from gawk, whose `intdiv(num, den, arr)` takes a result array instead of returning the quotient.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

intdiv0

```
intdiv0(a, b)
```

Same as `intdiv` but yields 0 instead of failing when `b` is zero (gawk `intdiv`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

strtonum

```
strtonum(s)
```

Numeric value of `s`, honoring 0x/0 prefixes (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

rand

```
rand()
```

Pseudo-random number in `[0, 1)`.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

srand

```
srand([x])
```

Seed the RNG with `x` (or time); returns the previous seed.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

I/O and General Functions

6 topics

print

`print` — keyword

Write its arguments to output, separated by `OFS` and terminated by `ORS`.

printf

```
printf(fmt, ...)
```

Format and print the arguments per `fmt`.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

getline

`getline` — keyword

Read the next record into `$0` or a variable from input, a file, or a command.

close

```
close(file [, how])
```

Close an open file/pipe; returns its status.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

fflush

```
fflush([file])
```

Flush buffers for `file`, or all outputs if omitted.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

system

```
system(cmd)
```

Run `cmd` via the shell; returns its exit status.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Blocked under [-5/--sandbox](#).

Time Functions

6 topics

sys`time`

```
systime()
```

Current time as seconds since the epoch (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

str`ftime`

```
strftime([fmt [, ts [, utc]])
```

Format timestamp `ts` per `fmt` (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

mk`time`

```
mktime(spec [, utc])
```

Convert a "YYYY MM DD HH MM SS [DST]" spec to a timestamp (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

get`timeofday`

```
gettimeofday(arr)
```

Clear `arr` and set `arr["sec"]` to the fractional epoch seconds and `arr["usec"]` to the microsecond remainder; returns 0 (gawk `time`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

getlocaltime

```
getlocaltime(arr [, ts])
```

Clear **arr** and fill it with the broken-down local-time fields **sec**, **min**, **hour**, **mday**, **mon** (1-12), **year** (full), **wday**, **yday** (1-based), **isdst**; returns the epoch seconds used. Uses the current time when **ts** is omitted (gawk **time**).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the **AWKRS_FUSEVM_NATIVE=1** backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under **-P/--posix** and **-c/--traditional**.

sleep

```
sleep(sec)
```

Sleep for **sec** seconds, fractions included; a negative duration is a fatal error. Returns 0 (gawk **time**).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the **AWKRS_FUSEVM_NATIVE=1** backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under **-P/--posix** and **-c/--traditional**.

Bitwise Functions

6 topics

and

```
and(v1, v2, ...)
```

Bitwise AND of the arguments (gawk).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkAnd` when the default numeric-chunk offload takes the surrounding chunk. The `AWKRS_FUSEVM_NATIVE=1` backend has no op for it and refuses to compile a program that calls it.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

or

```
or(v1, v2, ...)
```

Bitwise OR of the arguments (gawk).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkOr` when the default numeric-chunk offload takes the surrounding chunk. The `AWKRS_FUSEVM_NATIVE=1` backend has no op for it and refuses to compile a program that calls it.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

xor

```
xor(v1, v2, ...)
```

Bitwise XOR of the arguments (gawk).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkXor` when the default numeric-chunk offload takes the surrounding chunk. The `AWKRS_FUSEVM_NATIVE=1` backend has no op for it and refuses to compile a program that calls it.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

compl

```
compl(v)
```

Bitwise complement of `v` (gawk).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkComplJit` when the default numeric-chunk offload takes the surrounding chunk. The `AWKRS_FUSEVM_NATIVE=1` backend has no op for it and refuses to compile a program that calls it.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

lshift

```
lshift(v, n)
```

v left-shifted by **n** bits (gawk).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkLshiftJit` when the default numeric-chunk offload takes the surrounding chunk. The `AWKR5_FUSEVM_NATIVE=1` backend has no op for it and refuses to compile a program that calls it.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

rshift

```
rshift(v, n)
```

v right-shifted by **n** bits (gawk).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkRshiftJit` when the default numeric-chunk offload takes the surrounding chunk. The `AWKR5_FUSEVM_NATIVE=1` backend has no op for it and refuses to compile a program that calls it.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

Array and Type Functions

6 topics

typeof

```
typeof(x)
```

Type of **x**: "scalar", "array", "untyped", ... (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

isarray

```
isarray(x)
```

1 if **x** is an array, else 0 (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

mkbool

```
mkbool(expr)
```

Boolean-typed value from the truth of **expr** (gawk).

Engine. Dispatched by awkrs's own VM. Also lowered to the native `fusevm::Op::AwkMkbool` when the default numeric-chunk offload takes the surrounding chunk. The `AWKRS_FUSEVM_NATIVE=1` backend has no op for it and refuses to compile a program that calls it.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

atsplit

```
atsplit(s, arr [, fpat [, seps]])
```

Split **s** into **arr** by the pattern **fpat** (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

asort

```
asort(src [, dst [, how]])
```

Sort array values; returns the element count (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

`asorti`

```
asorti(src [, dst [, how]])
```

Sort array indices; returns the element count (gawk).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Character and Text Functions

4 topics

chr

```
chr(n)
```

Single-character string for code point `n`; empty string when `n` is not a valid scalar value (gawk `ordchr`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

ord

```
ord(s)
```

Code point of the first character of `s`, or 0 when `s` is empty (gawk `ordchr`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

revoutput

```
revoutput(s)
```

Return `s` reversed by Unicode scalar. awkrs exposes gawk's `revoutput` demo as a plain function rather than an output wrapper (gawk `revoutput`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

revtwoway

```
revtwoway(s)
```

Return `s` reversed by Unicode scalar — identical to `revoutput`. awkrs exposes gawk's `revtwoway` demo as a plain function rather than a two-way coprocess (gawk `revtwoway`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

File and Directory Functions

11 topics

stat

```
stat(path, arr)
```

Clear `arr` and fill it with `type`, `size`, `dev`, `ino`, `mode`, `nlink`, `uid`, `gid`, `rdev`, `blksize`, `blocks`, `atime`, `mtime`, `ctime`; returns 0, or -1 with `ERRNO` set (gawk `filefuncs`). `type` is one of `file`, `directory`, `symlink`, `other`.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

statvfs

```
statvfs(path, arr)
```

Clear `arr` and fill it with the `f_bsize`, `f_frsize`, `f_blocks`, `f_bfree`, `f_bavail`, `f_files`, `f_ffree`, `f_favail`, `f_fsid`, `f_flag`, `f_namemax` filesystem fields; returns 0, or -1 on non-Unix platforms and errors (gawk `filefuncs`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

fts

```
fts(root, arr)
```

Walk `root` recursively and fill `arr[1]...arr[n]` with the sorted path list; returns the entry count, or -1 when `root` does not exist. awkrs returns a flat sorted path array rather than gawk's nested per-directory hierarchy (gawk `filefuncs`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

readdir

```
readdir(path, arr)
```

Clear `arr` and fill `arr[1]...arr[n]` with "name/type" strings, where type is `f`, `d`, `l`, or `u`; returns the entry count, or -1 with `ERRNO` set (gawk `readdir`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

readfile

```
readfile(path)
```

Whole contents of `path` as one string; empty string with `ERRNO` set on failure (gawk `readfile`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

rename

```
rename(old, new)
```

Rename `old` to `new`; returns 0 on success or -1 with `ERRNO` set (gawk `filefuncs`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

chdir

```
chdir(path)
```

Change the process working directory; returns 0 on success or -1 with `ERRNO` set (gawk `filefuncs`). Blocked under `-S/--sandbox`.

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

inplace_tmpfile

```
inplace_tmpfile(path)
```

Create and return a unique sibling temp path next to `path` (`.NAME.awkrs_inplace.NANOS`) for the edit-then-rename cycle; empty string with `ERRNO` set on failure (gawk `inplace`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

inplace_commit

```
inplace_commit(tmp, dest)
```

Atomically rename `tmp` over `dest`, completing an in-place edit; returns 0 or -1 with `ERRNO` set (gawk `inplace`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

`writea`

```
writea(file, arr)
```

Write `arr` to `file` in awkrs's own `awkrs-rwarray-v1` text format (a magic line, then tab-separated escaped key/value lines); returns 0 or -1. Not binary-compatible with gawk's `rwarray` (gawk `rwarray`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

`reada`

```
reada(file, arr)
```

Clear `arr` and reload it from an `awkrs-rwarray-v1` file written by `writea`; returns 0, or -1 when the file is missing or carries the wrong magic line (gawk `rwarray`).

Engine. Dispatched by awkrs's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`, and blocked under `-S/--sandbox`.

Localization Functions

3 topics

bindtextdomain

```
bindtextdomain(domain, dirname)
```

Set `TEXTDOMAIN` to `domain`, point the catalog search at `dirname`, and load that catalog if present; returns `dirname`. Both arguments are required, and `awkrs` takes them in `(domain, dirname)` order.

Engine. Dispatched by `awkrs`'s own VM. Neither `fusevm` route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

dcgettext

```
dcgettext(string, domain, category)
```

Translate `string` through the loaded `domain` catalog, returning `string` unchanged when no catalog is loaded. All three arguments are required and `category` is accepted but not used for lookup.

Engine. Dispatched by `awkrs`'s own VM. Neither `fusevm` route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

dcngettext

```
dcngettext(s1, s2, n, domain, category)
```

Plural-aware translation of `s1/s2` for count `n` through the `domain` catalog; with no catalog loaded, returns `s1` when `n` is 1 and `s2` otherwise. All five arguments are required and `category` is accepted but not used for lookup.

Engine. Dispatched by `awkrs`'s own VM. Neither `fusevm` route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Restrictions. Rejected as a non-POSIX extension under `-P/--posix` and `-c/--traditional`.

The Intercept Engine

9 topics

before advice

```
intercept("before", pattern, code)
```

Runs `code` before every matching user-function call, then lets normal dispatch run the original. Every matching **before** advice fires, in registration order. If only **before** advice matched, the call proceeds untouched — the advice cannot change the arguments or the result.

after advice

```
intercept("after", pattern, code)
```

Runs `code` after the original returns. The presence of **after** advice makes the intercept machinery call the original itself, so `INTERCEPT_MS` / `INTERCEPT_US` are populated with the measured duration before the advice runs. The original's return value reaches the caller unchanged.

around advice

```
intercept("around", pattern, code)
```

Wraps the call. The original does not run unless `code` calls `intercept_proceed()`. At most one **around** advice is honored per call — the first match wins. The value the caller receives is whatever `intercept_proceed()` captured, or the awk empty value when the advice never proceeded; the advice's own `return` value is discarded, so **around** advice can suppress a call but cannot rewrite its result.

intercept patterns

```
intercept(kind, "draw_*", code)
```

A pattern matches by exact function name, or as a shell-style glob (*** for any run including empty, *?* for one character) against both the bare function name and the `"name arg1 arg2"` call string. *"*"* and *"all"* match every call. Any other character, including `[`, matches literally — there are no bracket expressions.

INTERCEPT_NAME

```
INTERCEPT_NAME
```

Global exposed to advice for the span of an intercepted call: the name of the function being intercepted. Removed again when the call completes.

INTERCEPT_ARGS

```
INTERCEPT_ARGS
```

The call's arguments joined with single spaces, in their string forms. A lossy view — use it for logging and matching, not for reconstructing values.

INTERCEPT_CMD

INTERCEPT_CMD

INTERCEPT_NAME and INTERCEPT_ARGS joined by a space, or just the name when the call had no arguments. This is the string glob patterns are matched against.

INTERCEPT_MS

INTERCEPT_MS

Wall-clock duration of the intercepted call in milliseconds, formatted to three decimals. Set only before *after* advice runs, so *before* and *around* advice never see it.

INTERCEPT_US

INTERCEPT_US

The same measurement in whole microseconds. Set alongside INTERCEPT_MS, with the same *after*-advice-only visibility.

Intercept Functions

5 topics

intercept

```
intercept(kind, pattern, code)
```

Register AOP advice (*kind*: "before"/"after"/"around") on user-function calls matching *pattern*; returns the ID (awkr's extension).

Engine. Dispatched by awkr's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

intercept_proceed

```
intercept_proceed()
```

From around advice: run the original function and return its value (awkr's extension).

Engine. Dispatched by awkr's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

intercept_list

```
intercept_list()
```

List registered intercepts (to stderr); returns the count (awkr's extension).

Engine. Dispatched by awkr's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

intercept_remove

```
intercept_remove(id)
```

Remove the intercept with *id*; returns 1 if removed, else 0 (awkr's extension).

Engine. Dispatched by awkr's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

intercept_clear

```
intercept_clear()
```

Remove all intercepts; returns the number cleared (awkr's extension).

Engine. Dispatched by awkr's own VM. Neither fusevm route lowers it: the numeric-chunk offload treats the chunk as ineligible, and the `AWKRS_FUSEVM_NATIVE=1` backend refuses to compile a program that calls it rather than lowering it to something approximate.

Special Variables

28 topics

NR

NR — special variable

Total number of input records read so far.

NF

NF — special variable

Number of fields in the current record.

FNR

FNR — special variable

Record number within the current input file.

FILENAME

FILENAME — special variable

Name of the current input file.

FS

FS — special variable

Input field separator (default " ").

OFS

OFS — special variable

Output field separator (default " ").

ORS

ORS — special variable

Output record separator (default "\n").

RS

RS — special variable

Input record separator (default "\n").

RT

RT — special variable

Text matched by RS for the current record (gawk).

SUBSEP

SUBSEP — special variable

Subscript separator for multi-dimensional array keys.

RSTART

RSTART — special variable

Start position of the last match() (1-based), or 0.

RLENGTH

RLENGTH — special variable

Length of the last match(), or -1.

CONVFMT

CONVFMT — special variable

Conversion format for numbers used as strings (default "%.6g").

OFMT

OFMT — special variable

Output format for numbers in print (default "%.6g").

FPAT

FPAT — special variable

Regex describing field contents, as an alternative to FS (gawk).

FIELDWIDTHS

FIELDWIDTHS — special variable

Space-separated fixed field widths for parsing (gawk).

IGNORECASE

IGNORECASE — special variable

When non-zero, regex and string comparisons ignore case (gawk).

ARGC

ARGC — special variable

Count of command-line arguments in ARGV.

ARGV

ARGV — special variable

Array of command-line arguments.

ARGIND

ARGIND — special variable

Index in ARGV of the current input file (gawk).

ENVIRON

ENVIRON — special variable

Array of the process environment variables.

ERRNO

ERRNO — special variable

Description of the last getline/close/system error (gawk).

PROCINFO

PROCINFO — special variable

Array of process/runtime information (gawk).

SYMTAB

SYMTAB — special variable

Array aliasing the program's global variables (gawk).

FUNCTAB

FUNCTAB — special variable

Array of the program's function names (gawk).

BINMODE

BINMODE — special variable

Binary I/O mode control (gawk).

LINT

LINT — special variable

Dynamic control of lint warnings (gawk).

TEXTDOMAIN

TEXTDOMAIN — special variable

Text domain for string translation (gawk).

Keywords and Control Flow

21 topics

BEGIN

BEGIN — keyword

Special pattern whose action runs once before any input is read.

END

END — keyword

Special pattern whose action runs once after all input is consumed.

BEGINFILE

BEGINFILE — keyword

Pattern whose action runs before each input file is read (gawk).

ENDFILE

ENDFILE — keyword

Pattern whose action runs after each input file is processed (gawk).

function

function — keyword

Define a user function: **function** *name(params)* { *body* }.

return

return — keyword

Return from a user function, optionally with a value: **return** [*expr*].

if

if — keyword

Conditional statement: **if** (*cond*) *stmt* with an optional **else** branch.

else

else — keyword

The alternative branch taken when an **if** condition is false.

while

while — keyword

Loop that repeats *stmt* while *cond* is true: **while** (*cond*) *stmt*.

do

do — keyword

Do-while loop: **do** *stmt* **while** (*cond*); the body runs at least once.

for

for — keyword

Loop: **for** (*init*; *cond*; *incr*) *stmt*, or **for** (*key* in *array*) *stmt*.

in

in — keyword

Array membership test (*key* in *arr*) or iteration (**for** (*key* in *arr*)).

switch

switch — keyword

Multi-way branch on a value: **switch** (*expr*) { **case** ...: ... } (gawk).

case

case — keyword

A labeled branch inside a **switch** statement (gawk).

default

default — keyword

The fallback branch taken when no **case** matches in a **switch** (gawk).

break

break — keyword

Exit the innermost **for**, **while**, or **do** loop immediately.

continue

continue — keyword

Skip to the next iteration of the innermost loop.

next

next — keyword

Stop processing the current record and read the next one.

nextfile

nextfile — keyword

Stop processing the current input file and advance to the next.

`exit`

`exit` — keyword

Stop reading input and run `END`; `exit [expr]` sets the exit status.

`delete`

`delete` — keyword

Remove an array element (`delete arr[k]`) or clear it (`delete arr`).

Pattern Forms

6 topics

pattern { action }

```
pattern { action }
```

The rule form. For each input record, the pattern is evaluated and the action runs when it is true. A program is a sequence of these plus function definitions; there are no bare top-level statements.

{ action }

```
{ action }
```

Empty pattern — the action runs for every input record.

pattern (no action)

```
/error/
```

A rule with no action block prints `$0` whenever the pattern is true. `awkrs '/error/' log` is the grep-shaped form.

/regexp/

```
/regexp/ { action }
```

Regex pattern: true when the ERE matches `$0`. Equivalent to `$0 ~ /regexp/`, and subject to `IGNORECASE` the same way.

expression pattern

```
NF > 3 && $1 != "#" { action }
```

Any expression works as a pattern; the action runs when the expression is true by awk's truth rules (non-zero number, or non-empty string for values that are not numeric strings).

range pattern

```
pattern1, pattern2 { action }
```

Inclusive range: the rule turns on at the record where `pattern1` is true and off after the record where `pattern2` is true, both records included. Either side may be a regexp or an expression.

Operators

24 topics

=

```
lvalue = expr
```

Assignment; right-associative and the loosest-binding operator, so `a = b = 1` assigns 1 to both. The target may be a variable, a field, or an array element. The expression's value is the assigned value.

`+=` `-=` `*=` `/=` `%=`

```
lvalue += expr
```

Compound assignment: read the target, apply the binary operator, store the result. `/=` and `%=` raise the same fatal division-by-zero error as their binary forms.

`^=` `**=`

```
lvalue ^= expr
```

Compound exponentiation. `**=` is accepted as a synonym for `^=`; the lexer emits one token for either spelling.

`? :`

```
cond ? then : else
```

Conditional expression; only the selected arm is evaluated. Binds tighter than assignment and looser than `||`. Newlines directly after `?` or `:` are treated as whitespace, so a ternary may be split across lines. The else arm is parsed as an assignment expression, so `c ? x = 1 : x = 2` parses.

`||`

```
expr || expr
```

Logical or; short-circuits, yielding 1 or 0. A newline after `||` is whitespace, so a condition may be continued on the next line without a backslash.

`&&`

```
expr && expr
```

Logical and; short-circuits, yielding 1 or 0, and continues across a newline the same way `||` does. Binds tighter than `||`.

`in`

```
key in array
```

Membership test: 1 when `array[key]` exists, else 0. Unlike a plain subscript reference it does not create the element. Chains left to right at the same precedence level as the comparison operators.

`(i, j) in array`

```
(i, j) in array
```

Membership test for a multidimensional subscript: the index list is joined with `SUBSEP` and looked up as a single key. The parentheses are required — they are what distinguishes the form from a comma expression.

`== !=`

```
expr == expr
```

Equality and inequality. Both operands numeric (or numeric strings from input) compare numerically, so `$1 == 10` matches the field text `10.0`; a string literal operand forces a string comparison, which is why `"10" == 10` and `10 == "10"` can differ from a field comparison.

`< <= > >=`

```
expr < expr
```

Relational comparison, with the same numeric-versus-string rule as `==`. Inside a `print` or `printf` argument list a bare `>` is parsed as an output redirection, not a comparison — parenthesize the comparison to disambiguate.

`~`

```
expr ~ /regexp/
```

Regex match: 1 when the right operand, taken as an ERE, matches the left operand's string form. The right side may be a `/re/` literal, an `@/re/` typed regexp, or any string-valued expression used as a dynamic regexp.

`!~`

```
expr !~ /regexp/
```

Negated regexp match — 1 when `~` would yield 0.

concatenation

```
expr expr
```

Juxtaposition concatenates the operands' string forms; there is no operator symbol. It binds tighter than the comparisons and looser than `+/-`, which is why `print 1 " " 2+3` prints `1 5`.

`+ -`

```
expr + expr
```

Addition and subtraction on the operands' numeric values. Under `-M/--bignum` these evaluate at MPFR precision instead of `f64`.

`*`

```
expr * expr
```

Multiplication, numeric.

/

```
expr / expr
```

Division. A zero divisor is a fatal runtime error (**division by zero attempted**), not an infinity — on the fusevm path this is why awkrs lowers to `Op::AwkDiv` rather than fusevm's shared `Op::Div`, which yields an undefined value instead.

%

```
expr % expr
```

Remainder, taking the sign of the left operand (`-7 % 3` is `-1`). A zero divisor is fatal, with the error naming %.

unary - +

```
-expr
```

Numeric negation and the no-op unary plus, which forces a numeric coercion. Both bind looser than `^`, so `-2^2` is `-4`.

!

```
!expr
```

Logical negation: 1 when the operand is false by awk's truth rules, else 0.

^ **

```
expr ^ expr
```

Exponentiation, right-associative — `2^3^2` is `2^(3^2)`, or 512. `**` is accepted as a synonym. Postfix `++/--` are applied to the base before the exponent, so `x++^2` squares the pre-increment value.

++

```
++lvalue    lvalue++
```

Increment by one. The prefix form yields the new value, the postfix form the old one. Valid on variables, fields, and array elements.

--

```
--lvalue    lvalue--
```

Decrement by one, with the same prefix/postfix distinction as `++`.

\$

```
$expr
```

Field reference. `$0` is the whole record, `$1...$NF` the fields. Assigning to a field past `NF` extends the record and rebuilds `$0` with `OFS`; assigning to `$0` re-splits the record with the active field rule.

()

```
(expr)
```

Grouping. A (immediately after an identifier with no intervening space is lexed as a function call instead — POSIX awk's rule that `name(arg)` calls and `name (arg)` concatenates, which awkrs implements with a distinct token.

Redirection and getline Forms

13 topics

print > file

```
print expr-list > file
```

Write to `file`, truncating it the first time the program opens it and appending on every later write. Repeated `>` to the same name within one run does not re-truncate; `close(file)` before the next write does.

print >> file

```
print expr-list >> file
```

Same as `>` except the first open appends instead of truncating.

print | command

```
print expr-list | command
```

Run `command` through `sh -c` and write to its standard input. The pipe stays open until `close(command)` or program exit; the command string is the handle, so two redirections with the same text share one subprocess.

print |& command

```
print expr-list |& command
```

Two-way pipe (coprocess): the same `sh -c` command model, but both its standard input and standard output are connected. Read the other half back with `getline <& command`. Deadlocks are the caller's problem — flush or `close(command, "to")` before reading.

getline

```
getline
```

Read the next record from the main input into `$0`, updating `NF`, `NR`, and `FNR`. Returns 1 on a record, 0 at end of input, -1 on error.

getline var

```
getline var
```

Read the next main-input record into `var`, leaving `$0` and `NF` alone; `NR` and `FNR` still advance. Same 1 / 0 / -1 return protocol.

getline < file

```
getline < file
```

Read the next record from `file` into `$0`, updating `NF` but not `NR` or `FNR`. Returns -1 when the file cannot be opened, which is how a missing input is distinguished from an empty one (0).

getline var < file

```
getline var < file
```

Read from `file` into `var`, touching neither the fields nor the record counters. The plainest form: no side effects beyond `var` and `ERRNO`.

command | getline

```
command | getline
```

Run `command` through `sh -c` and read its next output line into `$0`, updating `NF` and `NR`. Loop with `while ((cmd | getline) > 0)` and finish with `close(cmd)`.

command | getline var

```
command | getline var
```

Read the command's next output line into `var`; `NR` advances, the fields do not change.

getline var <& command

```
getline var <& command
```

Read from the output half of the coprocess started by `|&` with the same command string. The `<&` spelling is a distinct token, so it is never confused with `<` followed by `&`.

/inet/tcp/lport/host/rport

```
print req |& "/inet/tcp/0/example.com/80"
```

A redirection target of this shape opens a TCP connection to `host:rport` instead of a file or a subprocess. A local port of 0 means an ephemeral client port; any other value is bound before connecting. Malformed paths — wrong field count, unparseable ports — are a runtime error, not a silent file open.

/inet/udp/lport/host/rport

```
print msg > "/inet/udp/0/198.51.100.7/514"
```

The same path grammar over a connected UDP socket: writes send datagrams and reads receive them. An `/inet/` path that is neither `tcp` nor `udp` is rejected with an error naming the two supported forms.

printf Conversions

12 topics

%d, %i

```
printf "%d\n", expr
```

Signed decimal integer; the value is truncated toward zero. Values outside `i64` fall back to a wider formatting path rather than wrapping, and under `-M/--bignum` the integer is taken from the MPFR value.

%u

```
printf "%u\n", expr
```

Unsigned decimal integer.

%o

```
printf "%o\n", expr
```

Unsigned octal. With the `#` flag the output carries a leading `0`.

%x, %X

```
printf "%x %X\n", expr, expr
```

Unsigned hexadecimal, lower- and upper-case. With the `#` flag the output is prefixed `0x` or `0X`.

%a, %A

```
printf "%a\n", expr
```

C99 hexadecimal floating point — `1.5` prints as `0x1.8p+0` (`%a`) or `0X1.8P+0` (`%A`). An exact, round-trippable rendering of the underlying double.

%f, %F

```
printf "%.2f\n", expr
```

Fixed-point decimal, six fraction digits by default.

%e, %E

```
printf "%e\n", expr
```

Scientific notation with six fraction digits by default; `%E` uses `E` for the exponent marker.

`%g, %G`

```
printf "%g\n", expr
```

Shorter of `%e` and `%f` for the value, with trailing zeros removed unless the `#` flag is given. This is the conversion `CONVFMT` and `OFMT` default to (`%.6g`).

`%c`

```
printf "%c\n", expr
```

A single character: the first character of a string argument, or the character for a numeric argument's code point. Padded with spaces even under the `0` flag, except in `--traditional` mode where the BSD awk zero-padding quirk is honored.

`%s`

```
printf "%.3s\n", expr
```

String. A precision truncates to that many characters. Like `%c`, the `0` flag does not zero-pad strings except under `--traditional`.

`%%`

```
printf "100%%\n"
```

A literal percent sign. Consumes no argument.

unknown conversions

```
printf "[%q][%s]\n", "x"
```

A conversion character outside the set above is emitted literally as `%q` and consumes no argument, so the following `%s` still receives the argument it was written for. The example prints `[%q][x]`.

printf Flags, Width, and Precision

12 topics

- (left justify)

```
printf "%-8s|\n", s
```

Left-justify within the field width instead of right-justifying.

+ (force sign)

```
printf "%+d\n", n
```

Always print a sign on a numeric conversion, + for non-negative values.

space (sign placeholder)

```
printf "% d\n", n
```

Print a leading space where a non-negative value's sign would go, so positive and negative values align. Ignored when + is also given.

(alternate form)

```
printf "%#x %#o\n", n, n
```

Alternate form: 0x/0X prefix for %x/%X, a leading 0 for %o, and retained trailing zeros for %g/%G.

' (thousands grouping)

```
printf "%'d\n", 1234567
```

Group the integer part with the locale's thousands separator. Under -N/--use-lc-numeric the separator comes from localeconv(); when the locale supplies none, the flag is a no-op rather than an error.

0 (zero pad)

```
printf "%05d\n", n
```

Pad numeric conversions with zeros rather than spaces. The zeros go between the sign and the digits, so %05d of -42 is -0042. Ignored for %s and %c outside --traditional.

field width

```
printf "%8s|\n", s
```

Minimum field width. Widths are capped at 100000 characters — beyond that the request is clamped instead of attempting the allocation.

***** (width from argument)

```
printf "%*d\n", 6, 42
```

Take the field width from the next argument. A negative width means left-justify, exactly as if `-` had been written.

.precision

```
printf "%.3f %.2s\n", x, s
```

Fraction digits for the floating conversions, significant digits for `%g`, maximum characters for `%s`. A bare `.` with no digits means precision zero.

.* (precision from argument)

```
printf "%.*f\n", 3, x
```

Take the precision from the next argument; a negative value is treated as zero.

N\$ (positional argument)

```
printf "%2$s %1$s\n", "a", "b"
```

Select the Nth argument explicitly instead of consuming the next one — the example prints `b a`. Useful when a translated format string needs a different argument order than the original.

h, l, L (length modifiers)

```
printf "%ld\n", n
```

Accepted and skipped. `awk` has one numeric type, so the C length modifiers carry no information; they are consumed so that formats copied from C source still work.

String Escape Sequences

13 topics

\n

```
"line\n"
```

Newline (0x0A).

\t

```
"a\tb"
```

Horizontal tab (0x09).

\r

```
"a\rb"
```

Carriage return (0x0D).

\a

```
"\a"
```

Alert / bell (0x07).

\b

```
"\b"
```

Backspace (0x08).

\f

```
"\f"
```

Form feed (0x0C).

\v

```
"\v"
```

Vertical tab (0x0B).

\\

```
"C:\\path"
```

A literal backslash.

`\"`

```
"say \"hi\""
```

A literal double quote inside a string literal.

`\/`

```
"a\/b"
```

A literal forward slash. Redundant in a string but accepted, so a regexp copied into a string literal keeps working.

`\xHH`

```
"\x41"
```

One or two hexadecimal digits as a code point — `\x41` is `A`. With no hex digit following, the `\x` is kept as a literal `x`.

`\NNN`

```
"\101"
```

One to three octal digits as a byte value — `\101` is `A`. Values above 0xFF are masked to a single byte.

`\c (unrecognized)`

```
"\q"
```

An escape that is not in this table drops the backslash and keeps the character, so `"\q"` is `q`. gawk warns about this under `--lint`; awkrs accepts it silently.

Source Directives and Inline Rust

6 topics

@include

```
@include "lib.awk"
```

Splice another source file in at this point, before lexing. Includes are resolved once per path — a file already pulled in is skipped rather than duplicated, so a diamond of includes does not redefine its functions.

@load

```
@load "filefuncs"
```

gawk's dynamic-extension directive. awkrs implements gawk's bundled modules — `filefuncs`, `readdir`, `time`, `inplace`, `ordchr`, `readfile`, `revoutput`, `revtwoway`, `rvarray`, `intdiv` — natively in Rust, so loading one of those names (with or without a `.so` suffix or a directory prefix) is accepted and ignored; nothing is `dlopened`. Any other name is treated as an `@include` of that `.awk` file.

@namespace

```
@namespace "util"
```

Set the default namespace for the rest of the file: unqualified identifiers are rewritten to `util::name` in the AST. Builtin names, the special globals, and function-local names are never prefixed, and a name that already contains `::` is left alone.

@/regexp/

```
r = @/^err/
```

A typed regexp constant — a first-class value that can be assigned, passed, and used on the right of `~`. `typeof()` reports `"regexp"`, which is what distinguishes it from an ordinary string used as a dynamic regexp.

@expr(args)

```
f = "handler"; @f(x)
```

Indirect function call: the callee name comes from the value of `expr`. The callee may be a variable, an array element (`@a[k](...)`), a field (`@$1(...)`), or a parenthesized expression. Parsed so that the argument list belongs to the indirect call, not to the callee expression.

rust { ... }

```
rust {  
  pub extern "C" fn triple(a: i64) -> i64 { a * 3 }  
}
```

Inline Rust FFI. A top-level `rust { ... }` block is rewritten before lexing into `BEGIN { __rust_compile("<base64>", <line> ) }`, which compiles the block and registers its exported functions. Those exports are then callable as barewords from awk. Resolution order at a call site is user awk functions, then language builtins, then the FFI registry — so an export can never shadow either. The `BEGIN` wrapper is required because awk's top level admits only `pattern { action }` rules.

PROCINFO Keys

26 topics

PROCINFO["version"]

```
PROCINFO["version"]
```

The awkrs package version.

PROCINFO["api"]

```
PROCINFO["api"]
```

Always the string `awkrs`. Where gawk reports its extension-API identity, awkrs names itself — the reliable way for a script to detect that it is running here.

PROCINFO["api_major"], PROCINFO["api_minor"]

```
PROCINFO["api_major"]
```

The extension-API version awkrs reports, 4 and 1.

PROCINFO["program"]

```
PROCINFO["program"]
```

The name the binary was invoked as.

PROCINFO["platform"]

```
PROCINFO["platform"]
```

`posix`, `mingw`, or `unknown` — gawk's vocabulary, deliberately not Rust's `std::env::consts::OS`, so a script testing for `posix` behaves the same here as under gawk.

PROCINFO["pid"], PROCINFO["ppid"]

```
PROCINFO["pid"]
```

Process and parent-process IDs. `ppid` is Unix-only.

PROCINFO["uid"], PROCINFO["euid"]

```
PROCINFO["euid"]
```

Real and effective user IDs (Unix only).

PROCINFO["gid"], PROCINFO["egid"]

```
PROCINFO["gid"]
```

Real and effective group IDs (Unix only).

PROCINFO["pgripid"]

```
PROCINFO["pgripid"]
```

Process group ID (Unix only).

PROCINFO["groupN"]

```
PROCINFO["group1"]
```

One key per supplementary group, numbered from `group1` in the order `getgroups` returns them (Unix only).

PROCINFO["errno"]

```
PROCINFO["errno"]
```

The numeric errno behind the most recent failed I/O, the counterpart to `ERRNO`'s message text.

PROCINFO["FS"]

```
PROCINFO["FS"]
```

Which field-splitting rule is currently active: `FS`, `FPAT`, `FIELDWIDTHS`, or `API` in CSV mode. Computed on each refresh from the live variables, not stored.

PROCINFO["strftime"]

```
PROCINFO["strftime"]
```

The format `strftime()` uses when called with no arguments; defaults to `%a %b %e %H:%M:%S %Z %Y`, gawk's `date(1)`-equivalent default.

PROCINFO["argv"]

```
PROCINFO["argv"][0]
```

A nested array of the full process command line, indexed from 0 — including the options that `ARGV` deliberately omits.

PROCINFO["identifiers"]

```
PROCINFO["identifiers"]["split"]
```

A nested array mapping every known name to `builtin`, `scalar`, `array`, or `user`. Built from the compiled program's slot, array, and function tables plus the builtin name list.

PROCINFO["mb_cur_max"]

```
PROCINFO["mb_cur_max"]
```

Maximum bytes per multibyte character in the current locale, best-effort.

PROCINFO["nproc"]

```
PROCINFO["nproc"]
```

Available CPU count. An `awkrs` addition, useful for choosing a `-j` value from inside a script that re-executes itself.

PROCINFO["sorted_in"]

```
PROCINFO["sorted_in"] = "@ind_num_asc"
```

Assignable: sets the traversal order for every subsequent `for (k in array)`. See the Array Traversal Order chapter for the accepted values. Defaults to the empty string, meaning unsorted.

PROCINFO["prec"], PROCINFO["roundmode"]

```
PROCINFO["prec"]
```

Working precision in bits and the MPFR rounding mode. Outside `-M/--bignum` the precision reads 53, the width of a double; the rounding mode defaults to `N` (nearest). Both are assignable before the values that depend on them are computed.

PROCINFO["prec_min"], PROCINFO["prec_max"]

```
PROCINFO["prec_min"]
```

MPFR's precision bounds. Present only under `-M/--bignum`.

PROCINFO["gmp_version"], PROCINFO["mpfr_version"]

```
PROCINFO["mpfr_version"]
```

Versions of the linked GMP and MPFR libraries, queried from the libraries themselves. Present only under `-M/--bignum`.

PROCINFO["pma"]

```
PROCINFO["pma"]
```

gawk's persistent-memory-allocator version. `awkrs` is not built with PMA, so this key is absent — matching a gawk built without it, rather than reporting a value that would be false.

PROCINFO["READ_TIMEOUT"]

```
PROCINFO["READ_TIMEOUT"] = 500
```

Read timeout in milliseconds. Initialized from the `GAWK_READ_TIMEOUT` environment variable when the script has not set it, and only when that value is positive.

PROCINFO[input, "READ_TIMEOUT"]

```
PROCINFO["-", "READ_TIMEOUT"] = 250
```

Per-input override of the read timeout, keyed by the input name joined with `SUBSEP`. One entry is pre-seeded for every file in `ARGV` plus `-` for standard input, each defaulting to the global timeout.

PROCINFO[input, "RETRY"]

```
PROCINFO["-", "RETRY"] = 1
```

Per-input retry flag, pre-seeded to 0 alongside the per-input read timeouts.

`PROCINFO["awkrs_binmode"]`

```
PROCINFO["awkrs_binmode"]
```

The current numeric value of `BINMODE`, mirrored here on each refresh. An awkrs key, prefixed so it can never collide with a future gawk one.

Array Traversal Order

13 topics

@unsorted

```
PROCINFO["sorted_in"] = "@unsorted"
```

Hash order — whatever the array's internal iteration produces. The default, and the fastest.

@ind_str_asc

```
PROCINFO["sorted_in"] = "@ind_str_asc"
```

By index, compared as strings, ascending.

@ind_str_desc

```
PROCINFO["sorted_in"] = "@ind_str_desc"
```

By index, compared as strings, descending.

@ind_num_asc

```
PROCINFO["sorted_in"] = "@ind_num_asc"
```

By index, compared as numbers, ascending.

@ind_num_desc

```
PROCINFO["sorted_in"] = "@ind_num_desc"
```

By index, compared as numbers, descending.

@val_str_asc

```
PROCINFO["sorted_in"] = "@val_str_asc"
```

By element value, compared as strings, ascending.

@val_str_desc

```
PROCINFO["sorted_in"] = "@val_str_desc"
```

By element value, compared as strings, descending.

@val_num_asc

```
PROCINFO["sorted_in"] = "@val_num_asc"
```

By element value, compared as numbers, ascending.

@val_num_desc

```
PROCINFO["sorted_in"] = "@val_num_desc"
```

By element value, compared as numbers, descending.

@val_type_asc

```
PROCINFO["sorted_in"] = "@val_type_asc"
```

By value type, ascending: uninitialized, then numbers, then strings, then subarrays.

@val_type_desc

```
PROCINFO["sorted_in"] = "@val_type_desc"
```

By value type, descending.

custom comparison function

```
PROCINFO["sorted_in"] = "my_cmp"
```

A bare identifier names a user function used as the comparator. It must take 2 parameters (the two indices) or 4 (index, value, index, value) and return a negative number, zero, or a positive number. A wrong arity is a runtime error naming the function. An unrecognized @... token falls back to unsorted with a one-time warning on standard error.

under --posix

```
awkrs --posix -f prog.awk
```

-P/--posix forces unsorted traversal regardless of what `PROCINFO["sorted_in"]` says — POSIX awk specifies no ordering, so the setting is ignored rather than rejected.

Command-Line Options

42 topics

`-f, --file`

```
awkrs -f prog.awk [-f more.awk] input
```

Read the program from a file. Repeatable; the sources are concatenated in order. A single `-f` with no other source-shaping flag is also the only form eligible for the compiled-bytecode cache.

`-F, --field-separator`

```
awkrs -F: '{ print $1 }' /etc/passwd
```

Set `FS` before the program runs. Accepts the attached form `-F:` as well as the separated one.

`-v, --assign`

```
awkrs -v n=3 '{ print $n }' input
```

Assign a global before `BEGIN` runs, with escape sequences processed. Repeatable, and the attached form `-vn=3` works.

`-e, --source`

```
awkrs -e 'BEGIN { print 1 }' -e 'END { print 2 }'
```

Add program text on the command line. Repeatable, and mixable with `-f`.

`-i, --include`

```
awkrs -i lib.awk -e 'BEGIN { helper() }'
```

Include a library file, the command-line equivalent of `@include`. Repeatable.

`-l, --load`

```
awkrs -l mylib -e 'BEGIN { f() }'
```

Load an extension by name, resolved against `AWKPATH` (default `.`), trying `NAME.awk` then `NAME` in each directory. Not found is an error naming both candidates. Repeatable.

`-E, --exec`

```
#!/usr/bin/env -S awkrs -E
```

Read the program from a file and treat every remaining argument as data, never as an option. The safe form for `#!` scripts, since it stops a data filename that starts with `-` from being parsed as a flag.

-b, --characters-as-bytes

```
awkrs -b '{ print length($0) }' input
```

Treat input bytes as characters. `length`, `substr`, and `index` then count and slice bytes rather than Unicode scalars.

-c, --traditional

```
awkrs -c 'BEGIN { print length("x") }'
```

Traditional awk compatibility: the gawk-extension builtins are refused, and the BSD awk zero-padding quirk for `%0Ns` / `%0Nc` is enabled.

-P, --posix

```
awkrs -P -f prog.awk input
```

Strict POSIX mode: refuses the same extension builtins as `--traditional` and forces unsorted array traversal.

-n, --non-decimal-data

```
awkrs -n '{ print $1 + 0 }' hex.txt
```

Recognize `0x...` and leading-zero octal numbers in input data, not just in program text.

-M, --bignum

```
awkrs -M 'BEGIN { print 2 ^ 200 }'
```

Arbitrary-precision arithmetic through GMP/MPFR. Integer literals written without a decimal point keep their exact digits instead of rounding through a double, and `PROCINFO["prec"]` / `PROCINFO["roundmode"]` control the working precision.

-N, --use-lc-numeric

```
awkrs -N 'BEGIN { printf "%\x27d\n", 1234567 }'
```

Honor `LC_NUMERIC` in `printf/sprintf/print` output and in `CONVFMT/OFMT` formatting, including the `%` grouping flag. Input coercion is deliberately unaffected — `$1 + 0` still reads `.` as the radix point.

-k, --csv

```
awkrs --csv '{ print $2 }' data.csv
```

CSV mode: comma-separated with quoted fields and `"` as the embedded-quote escape. Reported as `PROCINFO["FS"] == "API"`.

-r, --re-interval

```
awkrs -r '/a{2,3}/' input
```

Accepted and ignored. `{m,n}` interval expressions are always available, so the flag exists only so that old command lines keep working.

-O, --optimize

```
awkrs -O -f prog.awk input
```

Accepted for gawk compatibility. Optimization and the JIT are on by default, so this changes nothing; `-s` is what turns them off.

`-s, --no-optimize`

```
awkrs -s -f prog.awk input
```

Disable optimization and the JIT, forcing the plain interpreter. The first thing to try when a program's results differ between runs — if `-s` changes the answer, the bug is in a compiled path.

`-j, --threads`

```
awkrs -j 8 '{ print $1 }' big.log
```

Worker threads for the parallel record engine (default 1). Only programs the parallel-safety analyzer accepts run in parallel; anything with cross-record state falls back to sequential execution.

`--read-ahead`

```
awkrs -j 4 --read-ahead 4096 '{ ... }'
```

Lines per batch when reading standard input in parallel mode (default 1024). Each batch is processed in parallel and printed in order before the next is read, so output ordering is preserved.

`-S, --sandbox`

```
awkrs -S -f untrusted.awk input
```

Disable `system()`, the file-I/O extension functions, pipes, coprocesses, and the network pseudo-paths. Each blocked call fails with a `sandbox:` runtime error rather than silently doing nothing.

`-L, --lint`

```
awkrs -L fatal -f prog.awk input
```

Enable lint warnings. The optional value selects `fatal`, `invalid`, or `no-ext` behavior.

`-t, --lint-old`

```
awkrs -t -f prog.awk input
```

Warn about constructs that will not port to old awk implementations.

`-d, --dump-variables`

```
awkrs -d vars.out -f prog.awk input
```

After the run, dump the final global variable state. Takes an optional path; with no value, or `-`, the dump goes to standard output.

`-D, --debug`

```
awkrs -D -f prog.awk input
```

List the program's rules and functions for debugging. Optional path; defaults to standard error.

`-o, --pretty-print`

```
awkrs -o formatted.awk -f prog.awk
```

Emit a re-indented program listing rebuilt from the AST. Optional path; defaults to standard output. The layout is awkrs's own — it is not byte-compatible with gawk's `--pretty-print`.

`-p, --profile`

```
awkrs -p prof.txt -f prog.awk input
```

Wall-clock summary with per-rule hit counts. Optional path; defaults to standard output. An awkrs-format report, not gawk's `awkprof.out`.

`-g, --gen-pot`

```
awkrs --gen-pot -f prog.awk
```

Scan the program for translatable strings and emit a `.pot` template.

`-I, --trace`

```
awkrs -I -f prog.awk input
```

Accepted for gawk CLI compatibility. Parsed and stored, but no opcode trace is emitted today — the flag has no runtime effect (like `-r/--re-interval`).

`-C, --copyright`

```
awkrs --copyright
```

Print the copyright notice and exit.

`-W`

```
awkrs -W version
```

mawk/BusyBox-style option bundle, comma-separated. `help/usage`, `version/v`, and `dump` act and exit; `exec=FILE` behaves like `-E`; `sprintf=N`, `posix_space`, `interactive`, and `random` are accepted and ignored so mawk command lines keep working.

`--repl`

```
awkrs --repl
```

Launch the interactive REPL. Also the default when awkrs is started on a terminal with no program and no input files.

`--lsp`

```
awkrs --lsp
```

Run as a Language Server over stdio: diagnostics, completion, hover, document symbols, signature help, goto-definition, references, highlights, and folding ranges. Nothing is written to the terminal — the process speaks JSON-RPC only.

--dap

```
awkrs --dap 127.0.0.1:4711 -f prog.awk input
```

Run as a Debug Adapter. With no value it speaks DAP over stdio; with `HOST:PORT` it connects to that address instead, which leaves the debugged program's own standard output free — the mode IDE plugins use.

--dump-tokens

```
awkrs --dump-tokens 'BEGIN { x = 1 + 2 }'
```

Print the lexer token stream, one `line<TAB>token` per line, and exit. Runs after `rust { }` desugaring, so an FFI block shows up as the `__rust_compile` call it becomes.

--dump-ast

```
awkrs --dump-ast 'BEGIN { x = 1 + 2 }'
```

Print the parsed AST and exit.

--dump-bytecode

```
awkrs --dump-bytecode 'BEGIN { x = 1 + 2 }'
```

Print the compiled bytecode ops, chunk by chunk, and exit.

--disasm

```
awkrs --disasm 'BEGIN { x = 1 + 2 }'
```

Print a fusevm disassembly listing — index, line, and mnemonic per op, with the chunk's name table — and exit. The readable counterpart to `--dump-bytecode`.

--tiers

```
awkrs --tiers -f prog.awk input
```

Run the program on the fusevm backend and then report which execution tier actually took each chunk: op count, whether the chunk was block-JIT eligible and compiled, the largest JIT-eligible region, and per-loop trace status. The answers come from fusevm's own predicates after the run, so this distinguishes **the JIT was enabled** from **the JIT compiled this**. A program outside the backend's coverage says so instead of reporting a tier.

--aot

```
awkrs --aot ./prog 'BEGIN { print 42 }'
```

Ahead-of-time compile a `BEGIN`-only program to a native executable at the given path, via a Cranelift object linked against the awk runtime.

-h, --help

```
awkrs --help
```

Print the help screen and exit.

-V, --version

```
awkis --version
```

Print the version and exit.

--

```
awkis -- '{ print }' -weird-filename
```

End option parsing. Everything after it is program text and input files, even if it begins with a dash.

Environment Variables

13 topics

AWKPATH

```
AWKPATH=/usr/local/share/awk:. awkrs -l mylib ...
```

Colon-separated search path for `-l/--load`, defaulting to `..`. Each directory is tried with `NAME.awk` and then bare `NAME`.

AWKRS_CACHE

```
AWKRS_CACHE=0 awkrs -f prog.awk input
```

Set to `0`, `false`, or `no` to disable the compiled-bytecode cache in `~/.awkrs/scripts.rkyv`. Any other value, or no value, leaves it enabled. The cache is keyed on the script's modification time and the `awkrs` binary's, so a rebuilt interpreter invalidates it automatically.

AWKRS_JIT

```
AWKRS_JIT=0 awkrs -f prog.awk input
```

Set to `0` to disable the JIT, the environment equivalent of `-s/--no-optimize`. `-s` wins regardless of this variable.

AWKRS_FUSEVM

```
AWKRS_FUSEVM=0 awkrs -f prog.awk input
```

Set to `0` to stop the interpreter offloading eligible numeric chunks to fusevm, forcing `awkrs`'s own opcode loop. On by default, and read once per process — changing it mid-run has no effect.

AWKRS_FUSEVM_NATIVE

```
AWKRS_FUSEVM_NATIVE=1 awkrs -f prog.awk input
```

Set to `1` to compile and run the whole program on the fusevm backend rather than the `vm.rs` interpreter. The validation harness for the ongoing migration: coverage is partial, and a construct the backend does not support is an error rather than a silent fallback.

AWKRS_AOT_RUNTIME_LIB

```
AWKRS_AOT_RUNTIME_LIB=/path/libawkrs_rt.a awkrs --aot ./prog ...
```

Override the runtime library the `--aot` linker step links against.

AWKRS_REPL_MODE

```
AWKRS_REPL_MODE=vi awkrs --repl
```

REPL edit mode, `emacs` or `vi` (`vim` is accepted for `vi`). Overrides the `[repl] mode` setting in `~/.awkrs/config.toml`; the default is `emacs`.

AWKRS_NO_CONFIG

```
AWKRS_NO_CONFIG=1 awkrs --repl
```

Set to any value to stop the REPL seeding a default `~/.awkrs/config.toml` on first launch. For CI and sandboxes that must not write to the home directory.

AWKRS_DAP_LOG

```
AWKRS_DAP_LOG=1 awkrs --dap -f prog.awk input
```

Set to any value to enable debug-adapter protocol logging.

GAWK_READ_TIMEOUT

```
GAWK_READ_TIMEOUT=500 awkrs -f prog.awk
```

Default read timeout in milliseconds, used to seed `PROCINFO["READ_TIMEOUT"]` when the script has not set it. Values are clamped to a non-negative 32-bit range, and a non-positive value leaves the key unset.

LC_NUMERIC

```
LC_NUMERIC=en_US.UTF-8 awkrs -N 'BEGIN { ... }'
```

Under `-N/--use-lc-numeric`, supplies the decimal point and thousands separator for output formatting. A locale with no thousands separator makes the `%'` flag a no-op.

LANGUAGE, LC_ALL, LC_MESSAGES, LANG

```
LANG=fr_FR.UTF-8 awkrs -f prog.awk
```

Consulted in that order to pick the gettext message catalog for `dcgettext` and `dcngettext`; the first one set wins.

NO_COLOR, CLICOLOR_FORCE

```
NO_COLOR=1 awkrs --help
```

Control the colorized help output. `NO_COLOR` set to anything disables color entirely; otherwise color is used when standard output is a terminal, or when `CLICOLOR_FORCE` is set.